

AI 활용 기술 문서

CADENCE — 매 틱마다 다음 순간을 설계하는 AI 디렉터

저장소 github.com/TaraGamesDev/cadence-nan2026 참가 개인

1. 요약

대부분의 “AI로 만든 게임”은 AI를 **개발 도구**로 씁니다. 이 프로젝트에서 AI는 **제품 안에서 돌아가는 시스템**입니다.

플레이어가 화면을 누를 때마다 — 초당 한 번꼴로 — 시스템은 그 한 번의 선택을 솔버의 최선수와 비교해 채점하고, 갱신된 실력 추정으로 **다음 순간이 어떤 난이도여야 하는지**를 정한 뒤, 그 의도를 만족하는 화살표 배치를 탐색해 보드에 놓습니다. 레벨을 미리 만들어 두지 않습니다. 매 순간을 그 자리에서 설계합니다.

런타임에 LLM 호출은 한 번도 일어나지 않습니다. 배포되는 게임은 전부 결정론적 코드이며 네트워크·API 키·계정이 필요 없습니다. LLM은 오프라인 정책 설계·튜닝 단계에만 사용했고, 그 산출물은 상수로 빌드에 구워져 있습니다. 자세한 사용 내역은 5장에 빠짐없이 기재했습니다.

2. 아키텍처

플레이어의 틱

- [스킬 추정기] `src/ai/skill.ts`
`regret = value(솔버의 최선수) - value(플레이어가 고른 수)`
→ $\theta(\text{숙련도})$, {계획성 · 효율 · 템포}, {좌절도 · 지루함}
- [AI 디렉터] `src/ai/director.ts` ← 의도
 $\theta + \text{감정} + \text{강도 램프} \rightarrow \text{플로우 채널 목표}$
계약조건 발행: 목표 난이도 / 스폰 수 / 탈출로 선호 깊이
/ 중앙 편향 / 방향 다양성
- [생성기 + 솔버] `src/ai/director.ts + src/ai/solver.ts` ← 시공
후보 배치 40개 샘플링 → 각각 솔버로 검증·채점 → 승자 선택
★ 불변식: 스폰 직후 반드시 최소 1수는 움직일 수 있다
- [글래스박스 패널] `src/ui/panel.ts`
위 전부를 화면에 그대로 노출

모듈	역할
core/grid.ts	보드 상태와 이동 규칙. 순수 함수, 할당 없음.
core/rng.ts	시드 기반 PRNG. 모든 확률적 결정이 여기서 나오므로 시드만 있으면 런 전체가 재현됩니다.
ai/solver.ts	국면 평가, 탈출 페포, 빔 서치 체인 탐색, 데드락 위험도.
ai/skill.ts	수순당 후회 → 숙련도 추정. 느리게 움직이는 실력과 빠르게 움직이는 감정을 분리.
ai/director.ts	제약조건 발행 + 제약 기반 스폰 생성.
ai/policy.ts	오프라인 튜닝으로 적합된 상수. 빌드에 구워짐.
ai/bots.ts	봇 페르소나. 콜드스타트 문제의 해법.

3. 핵심 설계 세 가지

3.1 수순당 후회(regret)로 실력을 잴다

“레벨을 클리어했는가”는 몇 분에 한 번 도착하고, 그 레벨이 마침 쉬웠는지에 오염됩니다. 대신 **모든 탭을, 그 탭이 이루어진 바로 그 국면에서** 채점합니다.

$$\text{regret} = \frac{(\text{최선수의 가치} - \text{플레이어가 고른 수의 가치})}{\text{그 국면에서의 선택지 가치 편차}}$$

국면이 통제되어 있으므로 “쉬운 판이라 잘한 것”과 “실제로 잘 둔 것”이 섞이지 않습니다. 선택지가 전부 동등한 국면은 **정보량 0**으로 처리해 추정을 흔들지 않습니다. 낫선 사람을 빠르게 파악해야 하므로 초반 학습률은 높고, 표본이 쌓이면 `samples / (samples + 10)` 비율로 사전확률에서 벗어납니다.

3.2 의도와 시공을 분리한다

디렉터는 화살표를 직접 놓지 않습니다. “**지금 이 순간이 어떻게 느껴져야 하는가**”를 제약조건으로 발행할 뿐이고, 실제 배치는 생성기가 샘플링하고 솔버가 검증합니다.

플레이어를 평가하는 쪽은 heuristic이어도 되지만, **보드를 건드리는 쪽은 틀리면 안 되기** 때문입니다. 추정이 어긋나면 난이도가 약간 안 맞을 뿐이지만, 배치가 잘못되면 게임이 망가집니다. 두 관심사를 코드 수준에서 분리해 두면 전자는 자유롭게 실험할 수 있고 후자는 정확히 검사할 수 있습니다.

3.3 약속은 작게, 대신 반드시 지킨다

초기 버전은 “**탈출 4수 보장**”을 약속했습니다. 벤치를 돌려보니 **1~2개 스폰으로는 구성 자체가 불가능한 약속**이었고, 60런에서 15,829회 위반했습니다. 그래서 선택으로 강등하고, 증명 가능한 것만 불변식으로 남겼습니다.

불변식 — 디렉터가 스폰한 직후, 반드시 최소 1수는 움직일 수 있다.

모든 후보 배치에 대해 **정확히** 검사합니다. 어떤 배치도 통과하지 못하면 스폰 수를 줄이지, 불변식을 깨지 않습니다. 플레이어는 자기 선택으로는 질 수 있어도, 디렉터 때문에 지지는 않습니다.

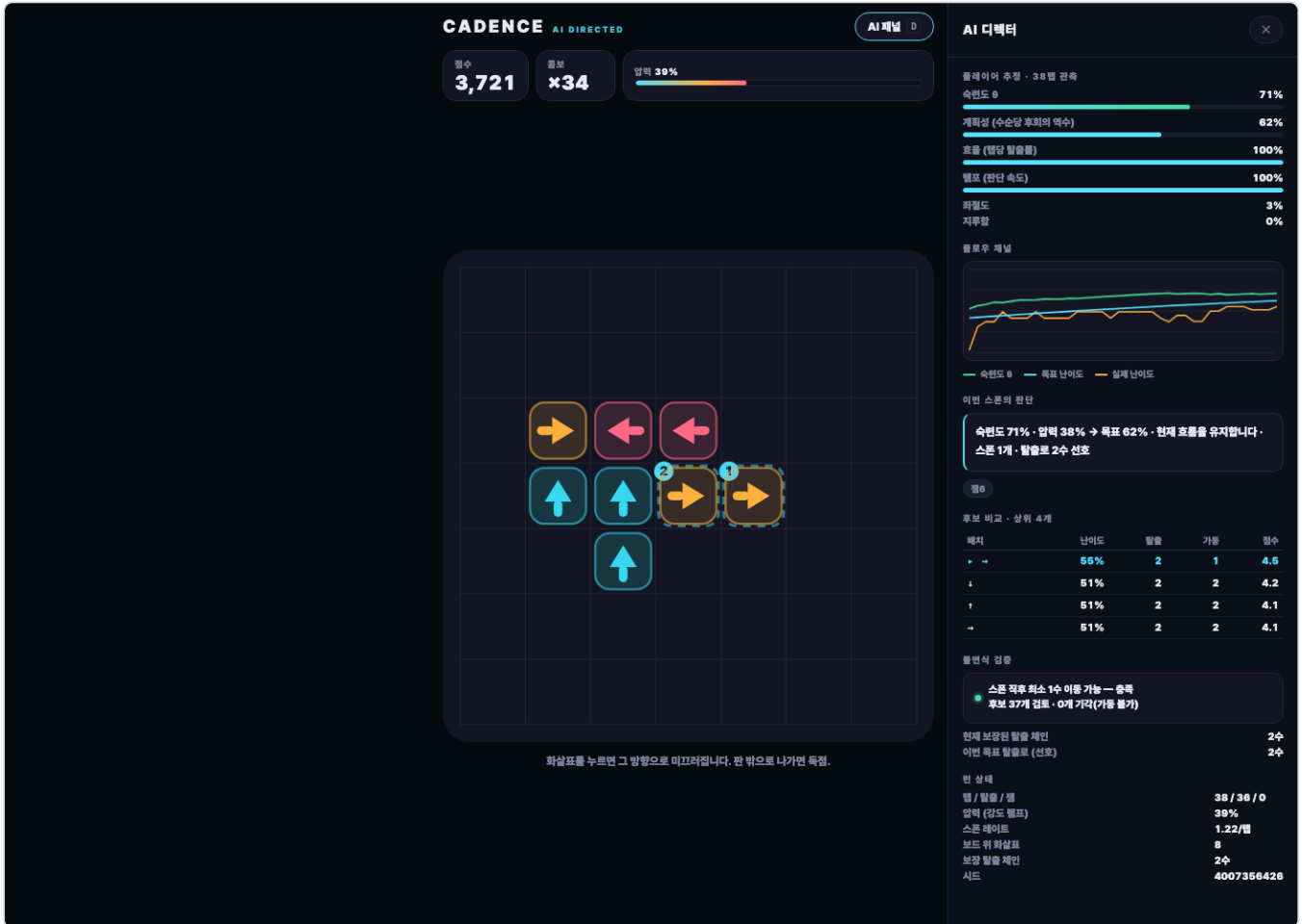
샘플링은 품질을 희생할 뿐 정확성을 희생하지 않습니다. 후보 공간은 (빈 칸 중 n 개 선택) $\times 4^n$ 이라 매 탭 전수 탐색이 불가능하지만, 살아남은 후보에 대한 불변식 검사는 근사가 아니라 실제 계산입니다.

보조 설계: 강도 램프

플로우 채널만으로는 런이 끝나지 않습니다. 초기 벤치에서 유능한 봇이 **4,000탭 동안 죽지 않고 4천만 점을 기록했습니다**. 디렉터가 플레이어를 너무 잘 살린 탓입니다. 그래서 실력과 무관하게 오르는 전역 강도 $\text{intensity} = 1 - e^{(-\text{탭}/77)}$ 를 도입했습니다. 플로우 채널이 **순간의 체감**을 잡고, 램프가 **런이 반드시 끝나게** 만듭니다.

4. 클래스박스 — AI가 자기 판단을 보여준다

조용히 도와주는 난이도 시스템은 자기 작업을 보여줄 수 있는 시스템보다 가치가 낮습니다. 그래서 디렉터가 마지막 결정에 사용한 모든 것이 화면에 있습니다.



실제 플레이 중 캡처. 좌: 보드 위 ①② 배지는 솔버가 증명한 탈출 체인입니다. 우: 38탭 관측 시점의 추정(θ 71%, 계획성 62%, 효율 100%), 플로우 채널 그래프, 이번 스킨의 판단 근거, 검토된 후보 4개와 채택되지 않은 대안들, 그리고 불변식 검증 결과 — "후보 37개 검토 · 0개 기각".

후보 비교표가 특히 중요합니다. 각 행은 실제로 평가된 배치이고, 열은 그 배치가 만들 난이도·보장 탈출 체인·가동 수·최종 점수입니다. 심사자는 AI가 무엇을 골랐는지뿐 아니라 무엇을 버렸는지까지 볼 수 있습니다.

5. AI 도구 사용 내역

사용한 도구와 활용 내역을 과장 없이 기재합니다.

단계	사용 도구	내용
런타임 (배포 빌드)	없음	LLM 호출 없음. 게임 안의 “AI”는 전부 결정론적 코드 — 솔버, 추정기, 디렉터. 네트워크·API 키·계정 불필요.
개발	Claude Code (<code>claude-opus-5</code>)	코딩 협업 도구로 사용. 게임 규칙 설계 토론, 엔진·AI·UI 구현, 헤드리스 벤치 작성, 설계 결함 진단. 아래 5.2 참조.
오프라인 튜닝	결정론적 탐색 (+ LLM 탐색 구현)	디렉터 정책 상수 적합. 현재 빌드에 반영된 값은 결정론적 탐색 결과입니다. LLM 유도 탐색도 동일 목적함수로 구현되어 있습니다 — 5.3 참조.
에셋 생성	없음	이미지·사운드·폰트를 생성하지도, 사용하지도 않았습니 다. 모든 그래픽은 Canvas API로 코드가 직접 그립니다.

5.1 런타임에 LLM을 쓰지 않은 이유

의도적인 설계 결정입니다. 세 가지 이유가 있습니다.

- **심사자가 링크만 눌러도 돌아가야 합니다.** 사전과제 요건은 “별도 유료 라이선스 없이 심사자가 실행할 수 있어야 한다”입니다. 런타임 LLM은 API 키를 요구하고, 키가 없으면 게임 자체가 죽습니다.
- **매 탭 = 초당 한 번의 판단입니다.** 이 빈도에서 네트워크 왕복은 게임을 성립시키지 못합니다. 실제 측정된 디렉터 지연은 탭당 평균 0.18ms입니다.
- **보드를 건드리는 코드는 틀리면 안 됩니다.** 3.2에서 설명한 분리 원칙에 따라, LLM은 *정책을 설계하는 자리에* 두고 *보드에 화살표를 놓는 자리*에는 두지 않았습니다.

5.2 개발 단계의 AI 활용

Claude Code를 코딩 협업 도구로 사용했습니다. 핵심 게임 규칙(“막히면 그 자리에 멈춘다”), 장르 선택, 스코프 결정은 사람이 내렸고, AI는 구현·진단·검증 자동화를 담당했습니다.

작업 방식에서 가장 중요했던 지시는 “**UI를 만들기 전에 헤드리스로 엔진을 검증한다**”였습니다. 이 한 가지가 세 개의 설계 결함을 화면에 닿기 전에 잡아냈습니다 (6장). 커밋 기록에 각 결함과 그 진단 과정이 남아 있습니다.

5.3 오프라인 튜닝의 LLM 프롬프트

`src/tools/tune.ts` 는 두 가지 탐색을 **같은 예산과 같은 목적함수**로 나란히 돌리도록 구현되어 있습니다: 균등 샘플링 (`random`) 과, 지금까지의 시행 결과를 보고 다음 후보를 제안하는 LLM 유도 탐색(`llm`). 둘을 동시에 보고하는 것이 요점입니다 — LLM 탐색만 돌리면 LLM이 *도움이 되었는지*를 말할 수 없습니다.

LLM 제안자에게 주는 시스템 프롬프트 전문입니다 (저장소 `src/tools/tune.ts`).

You are tuning the policy constants of a dynamic difficulty director for a real-time arrow puzzle game.

The director runs once per player tap. It estimates the player's skill from per-tap regret against a solver, derives a target challenge level, and then a generator samples candidate arrow spawns and picks whichever best matches that target. Your job is to choose the constants that govern the derivation.

Objective (lower loss is better). Loss combines, over three bot personas (novice, casual, expert):

- trackingError: |realised challenge - the director's own target|. The generator failing to deliver the director's intent.
- frustratedFraction / boredFraction: time spent outside the flow channel.
- session length outside 60..130 taps, penalised proportionally.
- MINUS 4x separation, where separation = expert's mean realised challenge - novice's. A director that gives everyone the same difficulty has failed.

Propose diverse candidates. Exploit what the trials show, but keep exploring - identical-looking candidates waste the budget. Respect every stated range.

모델은 `claude-opus-5` 이며, 응답은 구조화 출력(`output_config.format`)으로 JSON 스키마를 강제해 파싱 실패 가능성을 제거했습니다. 매 라운드 *지금까지의 최상위 시행 + 최근 시행*을 실제 수치와 함께 보내고 다음 후보 6개를 받습니다.

현재 빌드에 반영된 값은 결정론적 탐색의 결과입니다. LLM 유도 탐색은 구현·검증되어 있으나, 본 문서 작성 시점의 튜닝 실행에서는 실행 환경에 API 키가 설정되어 있지 않아 인증 단계에서 중단되었습니다. 키가 있으면 `npm run tune` 한 번으로 두 탐색의 비교가 재현됩니다. 과장하지 않기 위해 그대로 적습니다.

6. 검증

`npm run bench` 는 봇 페르소나로 전체 게임을 **실제 `tap()` 경로**를 통해 돌립니다. 심사자가 플레이하는 바로 그 코드에서 나온 수치입니다. 보고만 하는 것이 아니라 세 가지를 **단언**합니다.

페르소나	점수	탭	탈출	점	최고 콤보	θ	탭당 ms
novice	6,736	267.8	207.8	51.9	16.1	0.702	0.11
casual	9,312	115.3	103.3	5.9	40.8	0.801	0.17
expert	26,557	117.3	112.7	0.9	91.5	0.864	0.18

- OK — 불변식이 모든 런의 모든 탭에서 유지됨
- OK — 모든 런이 종료됨 (유능한 플레이어가 불사신이 되지 않음)
- OK — θ 가 세 페르소나를 마땅한 순서로 정렬 ($0.702 < 0.801 < 0.864$)

부수적으로 드러난 흥미로운 결과: 디렉터는 세션 길이를 **탭** 수가 아니라 **실시간 기준으로 균등화**합니다. 초보는 268탭 × 0.9초, 숙련자는 117탭 × 2.6초 — 둘 다 대략 **3~5분**입니다. 의도해서 넣은 규칙이 아니라 플로우 채널 추적에서 자연히 나온 성질입니다.

6.1 벤치가 화면에 닿기 전에 잡아낸 설계 결함

증상	원인과 수정
유능한 봇이 4,000탭까지 죽지 않음 (4천만 점)	종료 강제력 부재. 플로우 채널만으로는 런이 끝나지 않음 → 강도 램프 도입.
생존 보장 15,829회 위반	“탈출 4수 보장”이 애초에 구성 불가능한 약속 → 선호로 강등하고 증명 가능한 불변식 만 남김.
유능한 런의 하위 10%가 5탭 만에 종료	빈 보드를 데드락으로 오판. 화살표를 다 치우면 이기는 게 아니라 죽었음 → 종료 판정 수정 + 쿨드스타트 사전확률 도입(0가 1탭 만에 0.78로 튀어 처음부터 최대 압박이 걸림).
튜닝 1차 결과가 expert 239탭(약 10분)	길이 페널티가 분리도 보상에 비해 너무 약함 → 페널티 강화 후 전 페르소나 3~5분에 안착.

6.2 오프라인 튜닝 결과

후보 정책 48개 × 페르소나 3종 × 시드 6개로 전체 게임을 재생해 채점했습니다. 목적함수는 플로우 채널 추적 오차, 채널 밖 체류 시간, 세션 길이, 그리고 **초보와 숙련자가 실제로 겪는 난이도의 차이(분리도)**입니다.

정책	Loss (낮을수록 좋음)	분리도
손으로 정한 초기값	2.0815	0.0627
튜닝 결과 (빌드에 반영)	1.4564	0.0846

목적함수 기준 **30% 개선**, 분리도 **35% 향상**. 전체 시행 기록은 저장소 `tuning/result.json` 에 있습니다.

정직하게 덧붙이자면, expert 페르소나의 “지루함” 지표는 튜닝 후에도 0.66으로 높습니다. 다만 이 지표는 **후회가 낮고 탈출률이 높은 상태**로 정의되어 있어, 솔버를 그대로 따라가는 봇에게는 정의상 항상 참에 가깝습니다. 지표의 인공물로 보이며, 사람 플레이 데이터로 재검증이 필요한 부분입니다.

7. 외부 에셋 및 오픈소스 출처

외부 에셋 없음. 이미지·사운드·폰트·아이콘 파일이 저장소에 하나도 없습니다. 모든 그래픽(화살표, 타일, 그래프, 파티클)은 Canvas 2D API와 CSS로 코드가 직접 그립니다. 서체는 사용자 기기의 시스템 폰트를 사용합니다.

패키지	라이선스	범위	용도
<code>vite</code> ^6.0.5	MIT	devDependency	개발 서버 및 정적 빌드
<code>typescript</code> ^5.7.2	Apache-2.0	devDependency	타입 검사
<code>tsx</code> ^4.19.2	MIT	devDependency	벤치·튜너 실행
<code>@types/node</code>	MIT	devDependency	타입 정의
<code>@anthropic-ai/sdk</code>	MIT	devDependency	오프라인 튜너의 LLM 탐색 (5.3)

package.json의 **dependencies**는 비어 있습니다. 위 5개는 전부 개발 전용이며 배포 번들에 포함되지 않습니다. 빌드 산출물은 JS 23KB · CSS 8KB(gzip 각 10KB · 2KB)이고 외부 요청이 0건입니다.

8. 한계와 다음 단계

- **사람 데이터가 없습니다.** 정책은 봇 페르소나에 적합되어 있습니다. 봇은 사람의 대용이지 사람이 아닙니다 — 6.2의 “지루함” 지표 문제가 그 한계를 보여줍니다. 다음 단계는 실제 플레이 로그로 페르소나를 재적합하는 것입니다.
- **추정이 세션 내로 한정됩니다.** 계정·서버가 없으므로 0는 매 런 사전확률에서 다시 시작합니다. 세션 간 프로필을 유지하면 첫 30초의 추정 정확도가 크게 올라갑니다. 해커톤 스코프에서 의도적으로 제외했습니다.
- **후보 탐색이 샘플링입니다.** 매 탭 40개 후보는 지연 예산(0.18ms/탭) 안에서 정한 값입니다. 품질을 희생하지만 정확성은 희생하지 않습니다(3.3).
- **LLM 유도 탐색의 효용이 아직 미측정입니다.** 구현은 되어 있으나 5.3에 적은 이유로 비교 수치를 얻지 못했습니다.